Jakub Czermański 193105
Stanisław Pinkiewicz 193723

05.12.2025

DNN - Phase 3 report

1. Hyperparameters

The first thing to decide was how are we going to tune hyperparameters. We found two potential tools that do this: Optuna and WandB Sweeps.

Optuna is a well known framework and has many advantages

Ultimately, we chose WandB Sweeps as we are already using WandB and it's well integrated with it.

These are the steps we took to implement it:

- wrap the code with `train_sweep()` function
- create *sweep_configuration* variable which defines hyperparameters to be tested in a declarative way
- add support for config in build_model() function

2. Evidence of our efforts

Here is the chart provided to us from the laboratories instruction:

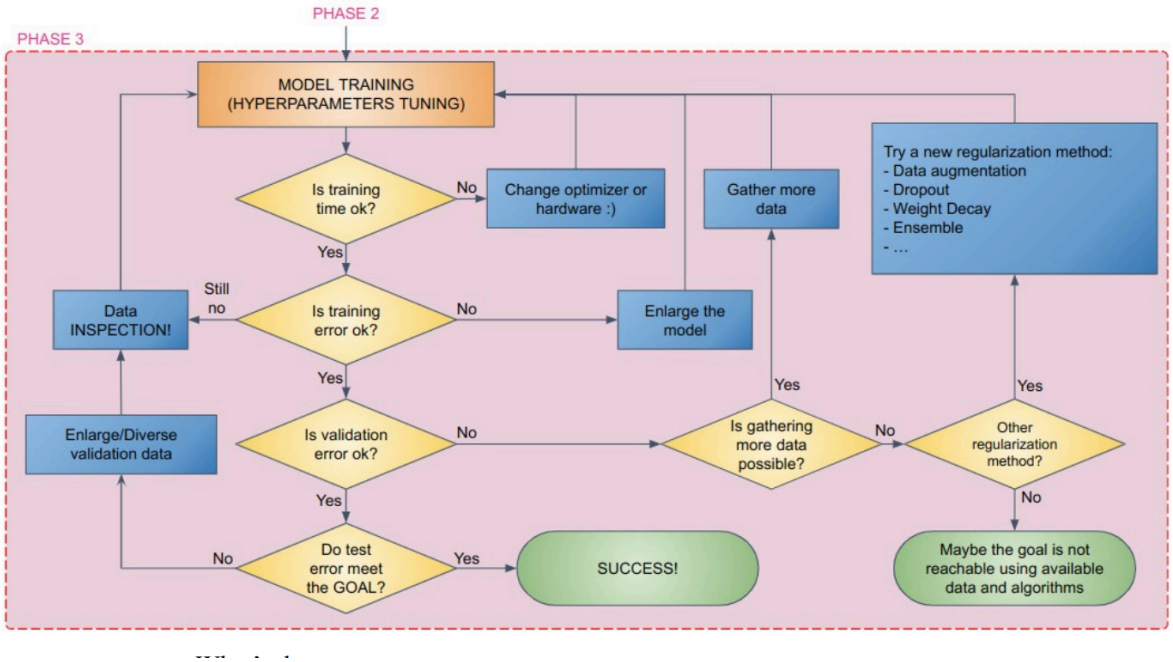


Figure 1: Steps required to achieve the goal

2.1. Training time

The first thing to do was to optimize the training time. To do this, we set up everything locally and removed code related to Colab as training there would take too much time. We used NVIDIA RTX 4060 GPU with CUDA enabled to ensure low training times in TensorFlow.

2.2. Training error & Validation error

Both training and validation errors were satisfactory as we had almost 100% accuracy on both the training and validation data sets while training. We tried to prevent overfitting on the validation and test sets by using data augmentation with `ImageDataGenerator()` function so the high accuracy means that our model is actually working quite well.

2.3. Test error

We had about 50% accuracy on test data. To solve it, we:

- checked again our source Kaggle notebook - it turned out its “test” data set was actually a validation data set. This means that we can’t rely on it working on the test data as well.
- researched other notebooks that could help us find out what we are doing wrong but didn’t find any really good ones
- we decided to determine the reasons behind low test accuracy ourselves.

2.4. Changing test data set (temporarily)

To verify that we’re doing everything correctly, we replaced our manually collected test data with data from Kaggle.

All code is provided in *changed_test_data.zip* file. We wrote the code in separate python files because it was easier to manage.

The results are shown on the following figures:

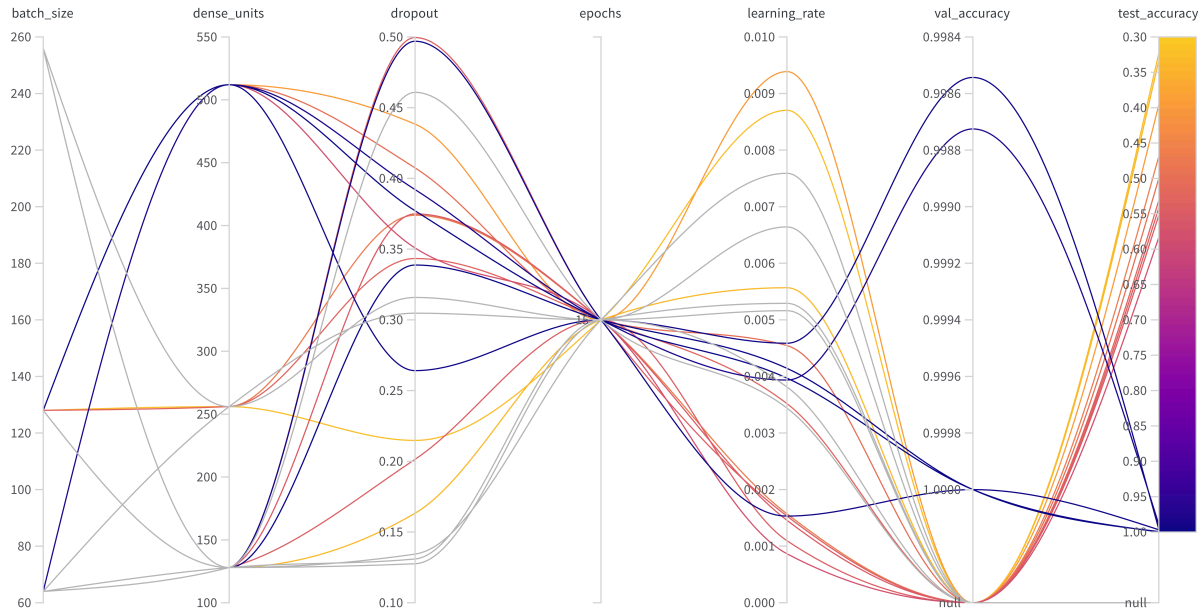


Figure 2: WandB parallel coordinates plot

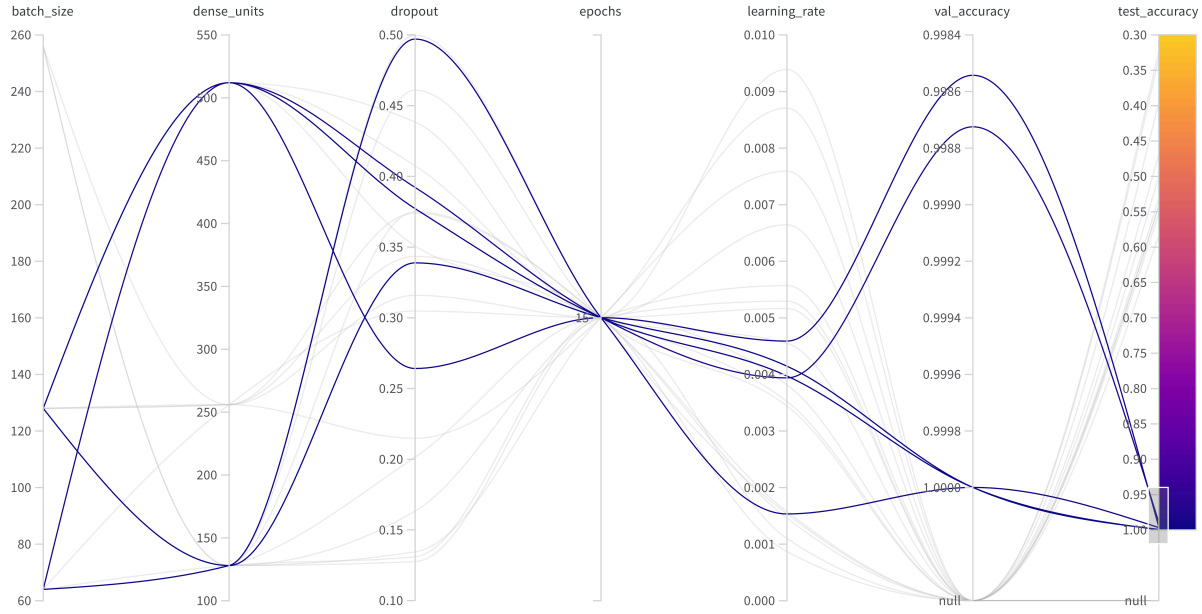


Figure 3: WandB parallel coordinates plot with best models selected

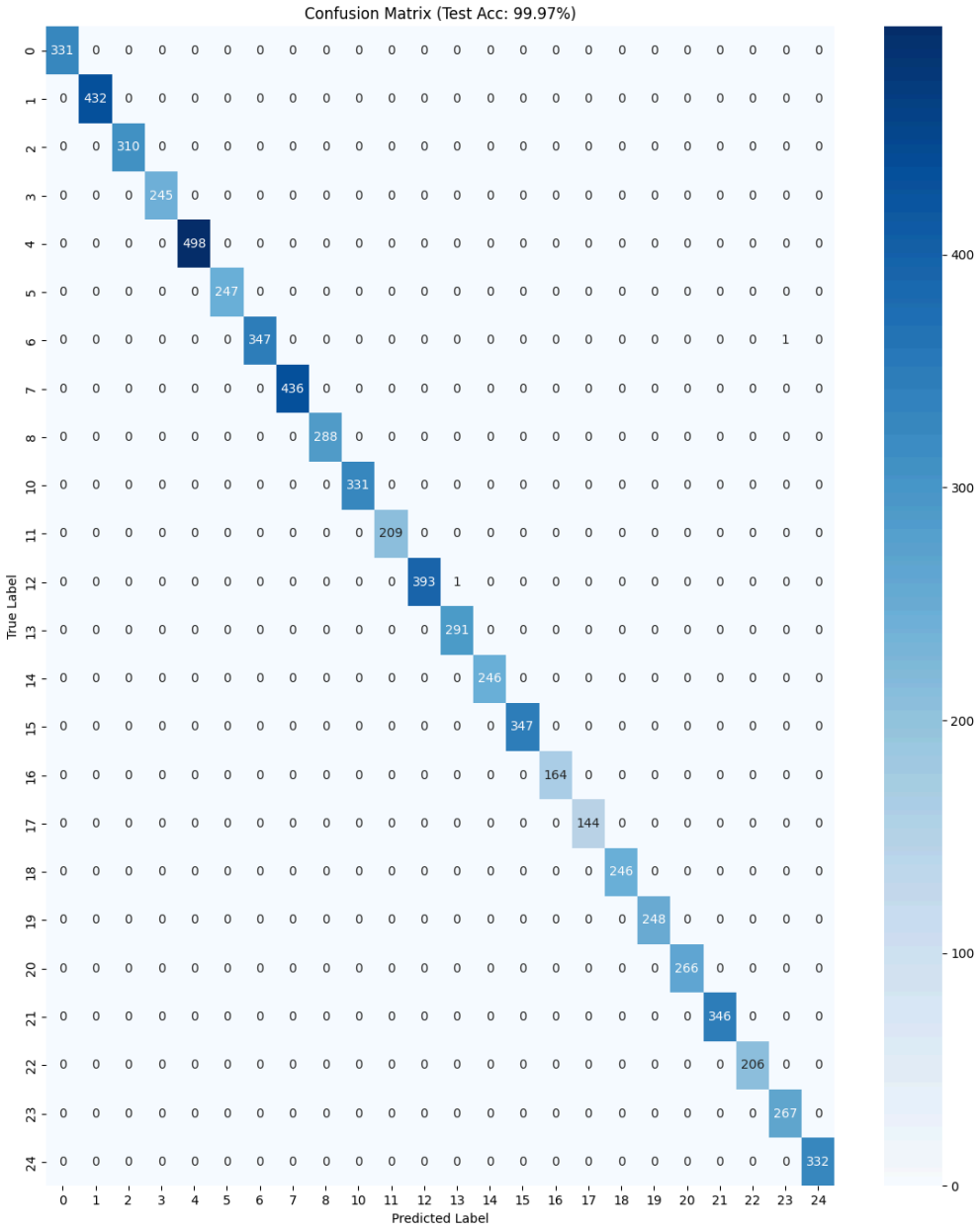


Figure 4: Confusion matrix showing the predictions of the best model

As we can see in Figure 3, the best models were able to predict results on both validation and test data sets with 100% accuracy.

This means that our code is working correctly. It is learned the patterns successfully and achieved almost perfect results on test data that was similar to the training and validation data (all data was from Kaggle). It however still struggles with our manually created images, even though we processed them to look the same as the ones in training data.

3. Final model and demo

We created a script that is showing abilities of our model. It is selecting the best model from the ones that we created, and uses it to classify one image, to mimic real life usage.

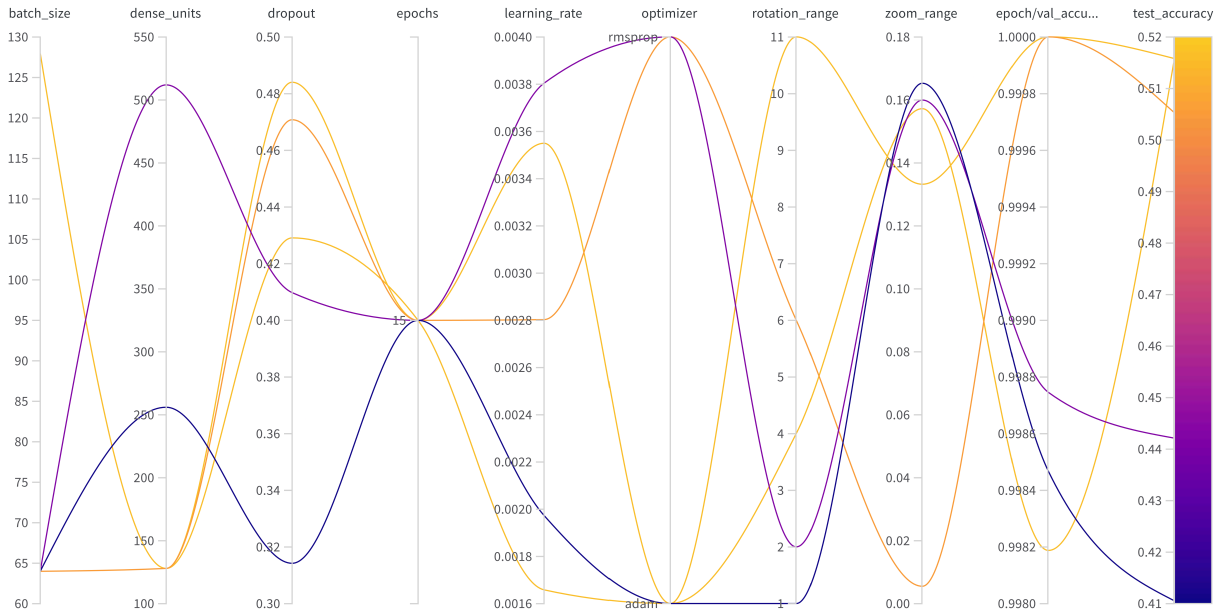


Figure 5: Parallel coordinates plot in WandB

```
sweep_configuration = {
    "method": "bayes",
    "metric": {"name": "val_accuracy", "goal": "maximize"},
    "parameters": {
        "epochs": {"value": 15},
        "batch_size": {"values": [64, 128]},
        "learning_rate": {"min": 0.0001, "max": 0.005},
        "optimizer": {"values": ["adam", "rmsprop"]},
        "dropout": {"min": 0.2, "max": 0.5},
        "dense_units": {"values": [128, 256, 512]},
        # Data Augmentation Params
        "rotation_range": {"min": 0, "max": 20},
        "zoom_range": {"min": 0.0, "max": 0.2}
    }
}
```

Hyperparameters are chosen from those ranges randomly to find configuration that gives best accuracy.

Logs are stored in *wandb* folder.

4. Conclusion

After performing multiple runs, and checking wide variety of hyper parameters we got to conclusion that the architecture of model we choose is good enough but not perfect. After more careful examination, it became apparent that either our photos are not clear enough or model is performing poorly outside of MNIST dataset. But examining photos didn't gave us any particular clues on what might be wrong with them. The most concerning thing about

the photos was that shadows present on the back of the wall might have been distracting for the model. But hue variation wasn’t big and that rather wasn’t a case.